HYBRID GNN BASED DRIVER MONITORING SYSTEM USING FACIAL LANDMARKS

A project report submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (AI & ML)

BY

M. POOJITHA	22BFA33175
M. SHOBHA RANI	22BFA33164
K. SAKETH RAM	22BFA33159
A. BHAGYARAJA	22BFA33141

Under the guidance of

Mrs. L Lavanya

Assistant Professor, CSE (AI&ML)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi& Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' grade)
Karakambadi Road, TIRUPATI – 517507

2022 - 2026

SRI VENKATESWARA COLLEGE OF ENGINEERING
(AUTONOMOUS)

**(Approved by AICTE, New Delhi& Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' Grade)
Karakambadi Road, TIRUPATI – 517507.**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

CERTIFICATE

This is to certify that the project work entitled, **"HYBRID GNN BASED DRIVER MONITORING SYSTEM USING FACIAL LANDMARKS"** is a Bonafide record of the project work done and submitted by

M. POOJITHA	**22BFA33175**
M.SHOBHA RANI	**22BFA33164**
K. SAKETH RAM	**22BFA33159**
A. BHAGYARAJA	**22BFA33141**

under my guidance and supervision for the partial fulfillment of the requirement for the award of B.Tech degree in **COMPUTER SCIENCE AND ENGINEERING (AI & ML).** This project is the result of our own effort and that it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above

GUIDE **HEAD OF THE DEPARTMENT**

External Viva-Voce Exam held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

We hereby declare that the project entitled **"HYBRID GNN BASED DRIVER MONITORING SYSTEM USING FACIAL LANDMARKS",** done by us under the esteemed guidance of **Mrs. L Lavanya,** Assistant Professor**,** CSE (AI&ML) and is submitted in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering (AI& ML).**

This report is the result of our own effort and it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above.

Date:

Place:

M. POOJITHA	**22BFA33175**
M. SHOBHA RANI	**22BFA33164**
K. SAKETH RAM	**22BFA33159**
A. BHAGYARAJA	**22BFA33141**

ACKNOWLEDGEMENT

We are thankful to our guide **Mrs. L Lavanya, Assistant Professor, Department of CSE (AI&ML)** for her valuable guidance and encouragement. Her helping attitude and suggestions have helped in the successful completion of the project report.

We would like to express our gratefulness and sincere thanks to **Dr. R. Swathi, Professor & HOD, Department of CSE (AI&ML),** for her kind help and encouragement during the course of study and in the successful completion of the project work.

We have great pleasure in expressing out hearty thanks to our beloved Principal **Dr. Vijaya Gunturu** for spending his valuable time with us to complete this project.

Successful completion of any project cannot be done without proper support and encouragement. We sincerely thank the management for providing all the necessary facilities during the course of study.

We would like to thank our parents and friends, who have the greatest contributions in all our achievements, for the great care and blessings in making us successful in all our endeavours.

M. POOJITHA **22BFA33175**
M. SHOBHA RANI **22BFA33164**
K. SAKETH RAM **22BFA33159**
A. BHAGYARAJA **22BFA33141**

ABSTRACT

Driver fatigue and stress are major causes of road accidents and unsafe driving conditions. Traditional methods that rely on Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) often fail under real-world conditions such as lighting variations, head movements, and subtle facial changes. Our project proposes a Hybrid Driver Monitoring System that detects both driver drowsiness and stress using facial landmarks. MediaPipe Face Mesh extracts 468 facial landmarks, classical features such as EAR and MAR detect fatigue, and a Graph Neural Network (GNN) analyzes the relationships between facial landmarks to identify drowsy and stress patterns. The system combines these outputs to determine the driver safety state (SAFE, WARNING, or CRITICAL) and triggers alerts in real time using a webcam.

CONTENTS

	Abstract	**i**
	Table of Contents	**ii-iii**
	List of Figures	**iv**
	List of Tables	**iv**
	Abbreviations	**v**
Chapter No	**Description**	**Page No**
1	**INTRODUCTION**	**1**
2	**PROJECT DESCRIPTION**	**2-3**
	2.1 Problem Definition	2
	2.2 Project Details	2-3
	2.3 Scope of the Project	3
3	**LITERATURE SURVEY**	**4-5**
	3.1 Related Work	4-5
4	**SYSTEM ANALYSIS**	**6-7**
	4.1 Existing System	6
	4.2 Disadvantages	6
	4.3 Proposed System	7
	4.4 Advantages	7
5	**SYSTEM REQUIREMENTS**	**8**
	5.1 Software Requirements	8
	5.2 Hardware Requirements	8
6	**SYSTEM DESIGN**	**9-16**
	6.1 System Architecture	9-10
	6.2 UML Diagrams	11-16
	6.2.1 Use Case Diagram	11-12
	6.2.2 Class Diagram	12-14
	6.2.3 Sequence Diagram	14-15
	6.2.4 Activity Diagram	15-16

CONTENTS

Chapter No	Description	Page No
7	**SYSTEM IMPLEMENTATION**	**17-19**
	7.1 Implementation Process	17
	7.2 Module Implementation	18-19
8	**SYSTEM STUDY AND TESTING**	**20-23**
	8.1 Feasibility Study	20
	8.2 Software Testing	21-23
9	**SOURCE CODE**	**24-31**
10	**SCREEN LAYOUTS**	**32-35**
11	**FUTURE ENHANCEMENTS**	**36**
12	**CONCLUSION**	**37**
13	**REFERENCES**	**38-39**

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
6.1	System Architecture	9
6.2.1	Use Case diagram	12
6.2.2	Class diagram	14
6.2.3	Sequence diagram	15
6.2.4	Activity diagram	16
10.1	Facial Landmark Detection using MediaPipe	32
10.2	Feature Extraction	32
10.3.1	Drowsiness Detection (ALERT)	33
10.3.2	Drowsiness Detection (DROWSY)	33
10.4.1	Stress Detection (LOW)	34
10.4.2	Stress Detection (MEDIUM)	34
10.5.1	Final Driver State Classification-1	35
10.5.2	Final Driver State Classification-2	35

.

LIST OF TABLES

Table No.	Table Name	Page No.
8.2.1	Test Cases for Driver Monitoring System	22-23

.

ABBREVIATIONS

AI	-	Artificial Intelligence
ML	-	Machine Learning
DL	-	Deep Learning
GNN	-	Graph Neural Network
GCN	-	Graph Convolution Network
CNN	-	Convolutional Neural Network
EEG	-	Electroencephalography
YOLO	-	You Only Look Once
EAR	-	Eye Aspect Ratio
MAR	-	Mouth Aspect Ratio
RAM	-	Random Access Memory
OpenCV	-	Open-Source Computer Vision Library
VS Code	-	Visual Studio Code
UML	-	Unified Modeling Language

Chapter 1

INTRODUCTION

Driver drowsiness and stress significantly affect driving performance and increase the risk of accidents. Fatigue reduces alertness and reaction time, while stress impacts concentration and decision-making ability. Traditional monitoring systems mainly detect eye closure or yawning, which are not always reliable in real driving environments. By using facial landmarks and Graph Neural Networks, the proposed system analyzes facial relationships and behavioral patterns more effectively. This approach enables real-time monitoring of both driver fatigue and stress levels, improving overall road safety.

Chapter 2

PROJECT DESCRIPTION

2.1 PROBLEM DEFINITION:

Driver fatigue and emotional stress are major contributors to road accidents worldwide. Existing driver monitoring systems mainly focus on detecting drowsiness using simple indicators such as eye blinking or yawning, which are sensitive to lighting conditions, head movement, and facial occlusions. Deep Learning models using raw images require high computational power, while EEG-based systems are intrusive and impractical for everyday driving. Therefore, there is a need for a light weight and real-time driver monitoring system capable of detecting both drowsiness and stress using facial features.

2.2 PROJECT DETAILS:

Our project, titled "Hybrid GNN Based Driver Monitoring System Using Facial Landmarks", aims to develop a real-time system that monitors driver condition and detects both drowsiness and stress using facial features.

The system uses a webcam to capture live video input of the driver. From each frame, facial landmarks are extracted using MediaPipe Face Mesh, which provides 468 key points representing different regions of the face. These landmarks are used to analyze facial behavior and expressions.

Our project follows a hybrid approach combining classical feature extraction and machine learning techniques. Classical features such as Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) are used to detect eye closure, blinking rate, and yawning patterns, which indicate driver fatigue.

In addition, a Graph Neural Network (GNN) model is used to analyze the relationships between facial landmarks by converting them into a graph structure. This helps in capturing complex facial patterns and improves the detection of both drowsiness and stress levels. Based on these analyses, the system classifies:

- Drowsiness as **ALERT or DROWSY**
- Stress as **LOW, MEDIUM, or HIGH**

Finally, a decision module combines both outputs to determine the overall driver state as **SAFE, WARNING, or CRITICAL**. If an unsafe condition is detected, the system generates alerts in real time.

This system is lightweight, non-intrusive, and suitable for real-world applications, as it works using a standard webcam without requiring additional hardware.

2.3 SCOPE OF THE PROJECT:

The scope of our project is to develop a real-time driver monitoring system that detects drowsiness and stress using facial landmarks through a webcam. The system continuously analyzes facial features to identify driver condition and improve road safety.

It can be used in vehicles to monitor driver's alertness without any additional hardware. The system provides alerts when unsafe conditions are detected. It works efficiently under normal conditions such as slight head movements and lighting variations.

Our project can be extended to smart vehicles, fleet monitoring systems, and real time safety applications. It can also be improved to detect driver distractions and integrated with advanced driver assistance systems.

Chapter 3

LITERATURE SURVEY

3.1 RELATED WORK

1 Varsha Agarwal, Rajneesh, "An Accurate Detection of Drowsiness Using a Graph-Based Neural Network."

This paper presents a driver drowsiness detection system using Graph Neural Networks (GNN). The authors use facial landmarks to construct graph structures and analyze spatial relationships for accurate detection. The study shows that GNN improves performance compared to traditional methods.

2 Neha Patil, Namrata Patole, Chetan Thorat, Abhishek Phapale, "Deep Learning Based Driver Yawning Detection."

This work focuses on detecting yawning using deep learning techniques. The system analyzes mouth opening patterns to identify fatigue. It demonstrates that yawning is an important indicator of driver drowsiness.

3 Quang Tran Ngoc, Seunghyun Lee, Byung Cheol Song, "Facial Landmark-Based Emotion Recognition via Directed Graph Neural Network."

This paper proposes emotion recognition using facial landmarks and directed GNN. It captures relationships between facial features to improve prediction accuracy, showing the effectiveness of graph-based learning.

4 Aryan Ritesh, Neel Jagatia, Pankaj Deshmukh, "Driver Drowsiness Detection Using Machine Learning."

This study uses machine learning techniques to detect drowsiness based on eye and facial movements. The system helps in early detection and improves road safety through timely alerts.

5 Mohammad Faizal, Nikhil, Prajwal, Aruna Rao B P, "Recognition of Stress Using Face Image and Facial Landmarks."

This study focuses on recognizing stress using facial features and image processing techniques. It shows that facial landmarks can be effectively used to estimate stress conditions.

6 Dr. R. M. S. Parvathi, M. Sanjay, P.S. Santhosh Kumar, P. Solai Murugan, "Stress Detection Using Machine Learning and Image Processing."

This paper presents a machine learning approach for stress detection using facial images. It combines image processing techniques with classification models to improve accuracy.

Chapter 4
SYSTEM ANALYSIS

4.1 EXISTING SYSTEM

The existing driver monitoring systems primarily rely on simple facial feature-based techniques to detect drowsiness. Common approaches use the Eye Aspect Ratio (EAR) to identify blinking patterns and the Mouth Aspect Ratio (MAR) to detect yawning. These methods are easy to implement but are highly sensitive to external conditions such as lighting variations, head movements, and facial occlusions.

Some advanced systems utilize Convolutional Neural Networks (CNN) for eye state classification, which improves accuracy to some extent. However, these models require large datasets and high computational resources, making them less suitable for real-time applications.

Most existing systems mainly focus on detecting drowsiness and often ignore the driver's stress levels, which also play a crucial role in driving safety. Additionally, certain systems use EEG-based monitoring techniques, which provide accurate results but are intrusive and not practical for everyday use.

4.2 DISADVANTAGES

Here are some common disadvantages of existing system:

- High false detection due to lighting variations
- Sensitive to head movements and face angle changes
- Cannot detect stress levels effectively
- Most systems focus only on drowsiness detection
- Poor performance in real-time conditions
- Requires high computational power (deep learning models)
- EEG-based systems are intrusive and not practical

4.3 PROPOSED SYSTEM

Our project proposes a real-time driver monitoring system that detects both drowsiness and stress using facial landmarks. The system uses a webcam to capture live video and applies MediaPipe Face Mesh to extract 468 facial landmarks.

These landmarks are used to compute classical features such as Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) for detecting fatigue-related patterns. In addition, a Graph Neural Network (GNN) model is used to analyze relationships between facial landmarks and identify complex patterns related to drowsiness and stress.

The system classifies the driver as ALERT or DROWSY and detects stress levels such as LOW, MEDIUM, or HIGH. A feature fusion and decision module combines these outputs to determine the final driver state as SAFE, WARNING, or CRITICAL.

If an unsafe condition is detected, the system generates real-time alerts to ensure driver safety.

4.4 ADVANTAGES

Here are advantages of the proposed system:

- Robust in real-world conditions
- Works with normal webcam
- Detects both driver drowsiness and stress levels
- Captures fine facial dynamics using facial landmarks
- Lightweight and suitable for real-time monitoring
- Hybrid approach improves accuracy and stability

Chapter 5

SYSTEM REQUIREMENTS

5.1 SOFTWARE REQUIREMENTS

- **Python:** Main programming language used for development.
- **OpenCV:** For video capture, image processing, and face detection tasks.
- **MediaPipe:** For accurate and fast facial landmark detection.
- **PyTorch:** Deep learning framework for training and running the GNN model.
- **PyTorch Geometric:** Specialized library for graph-based neural network operations.
- **VSCode /Jupyter Notebook:** For writing, testing, and running the code in a clean workflow environment.

5.2 HARDWARE REQUIREMENTS

- **Webcam:** A standard built-in or external webcam for real-time video input.
- **Processor:** Intel i5 /AMD Ryzen or higher to ensure smooth processing.
- **Memory (RAM):** Minimum 8 GB RAM for handling real-time detection and model operations.

Chapter 6

SYSTEM DESIGN

6.1 SYSTEM ARCHITECTURE:

The system architecture represents the workflow of the proposed driver monitoring system. It captures real-time video, extracts facial landmarks, and computes important features. These features are analyzed using GNN-based model to detect drowsiness and stress. The results are combined to determine the driver's condition and generate alerts when required.

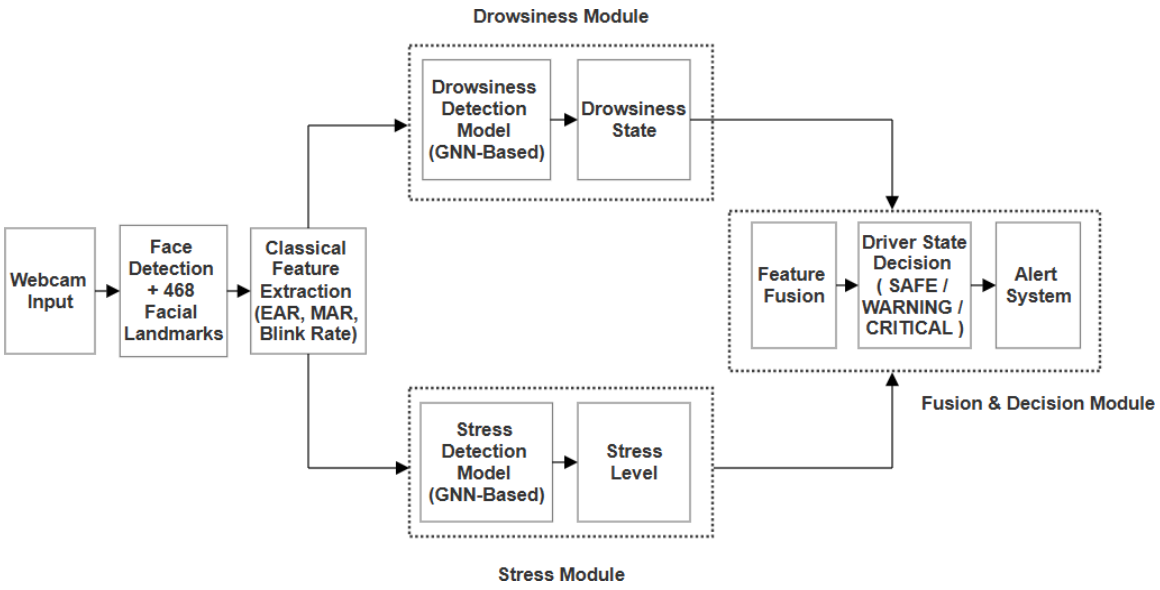


Fig 6.1: System architecture

1. Webcam Input: The webcam captures real-time video of the driver continuously. Each frame is processed for further analysis. It provides the input to the system for monitoring.

2. Face Detection & 468 Facial Landmarks: The system detects the driver's face in each frame. MediaPipe extracts 468 facial landmark points. These landmarks are used for feature extraction and analysis.

3. Classical Feature Extraction: Important features like EAR, MAR, and blinking patterns are computed from landmarks. These features help identify fatigue and expressions. They provide meaningful inputs for detection.

4. Drowsiness Detection Model: Facial landmarks are converted into a graph structure. A GNN is used to analyze relationships between facial points. The model detects patterns related to drowsiness.

5. Drowsiness State: The system classifies the driver as **ALERT or DROWSY**. This is based on eye closure and fatigue patterns. It indicates the driver's alertness level.

6. Stress Detection Model: Facial landmarks are converted into graph form. GNN analyzes facial relationships and expressions. The model detects stress patterns.

7. Stress Level: The system classifies stress as **LOW, MEDIUM, or HIGH.** This represents the emotional condition of the driver. It is used for further decision making.

8. Feature Fusion: The outputs from drowsiness and stress detection are combined. This helps improve accuracy and reliability. It ensures better overall results.

9. Driver State Decision: The system determines the final state as **SAFE, WARNING, or CRITICAL.** This is based on combined results. It represents the overall driver's condition.

10. Alert System: The system generates alerts when unsafe conditions are detected.

6.2 UML DIAGRAMS

What is UML?

The Unified Modeling Language (UML) is a standard graphical language used to visualize, design, and document the structure and behavior of software systems. It provides a set of diagrams that help in representing system architecture, components, and interactions in a clear and understandable manner. UML is widely used in software engineering to improve communication among developers, analyze system requirements,
and ensure proper system design before implementation.

In this project four basic UML diagrams have been explained:

1. Use Case Diagram
2. Class Diagram
3. Sequence Diagram
4. Activity Diagram

6.2.1 Use Case Diagram

The use case diagram represents the interaction between the system and the driver in the Driver Monitoring System. The process is initiated by the system through the Start Monitoring use case. Once monitoring begins, the system captures video frames and detects facial features and landmarks. From these, important features are extracted for further analysis.

The extracted features are used for both Drowsiness Detection and Stress Detection. These detection processes are included as part of the Classify Driver State use case, where the overall condition of the driver is determined. Based on the classified state, the system may generate alerts using the Generate Alert use case, which extends the classification process when necessary.

Finally, the driver interacts with the system by viewing alerts through the View Alert use case. This ensures that the driver is informed in real time about their condition, helping to improve safety and awareness while driving.

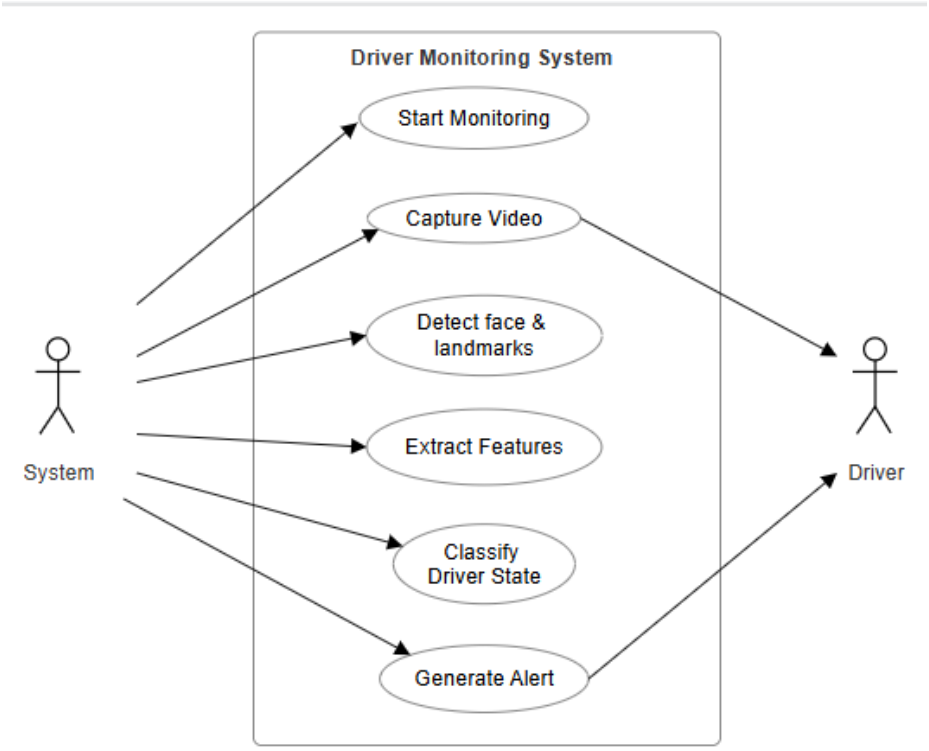


Fig 6.2.1: Use Case diagram

6.2.2 Class Diagram

The class diagram represents the structural design of the Driver Monitoring System, illustrating how different components interact to perform real-time driver state analysis. The central class, Driver Monitoring System, initiates the monitoring process and coordinates the overall workflow.

The Camera class captures video frames from the driver environment, which are then forwarded to the Face Detector class. The Face Detector identifies facial landmarks from the captured frames, enabling further analysis.

These landmarks are passed to the Feature Extractor, which computes important features such as Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR). These extracted features are essential inputs for detecting driver conditions.

The processed features are then provided to both the Drowsiness Detector and Stress Detector classes. Each of these modules utilizes a GNN-based model to analyze the features and detect drowsiness and stress levels respectively.

The outputs from both detectors are sent to the Determine Driver State class, which combines the results to evaluate the overall condition of the driver, such as alert, drowsy, or stressed.

Based on the determined driver state, the Alert System generates an alert (such as a warning sound) when a critical condition is detected. Finally, the Driver interacts with the system by viewing the generated alerts, ensuring timely awareness and improved safety.

This design ensures a clear and sequential flow of data between components, supporting efficient and real-time driver monitoring.

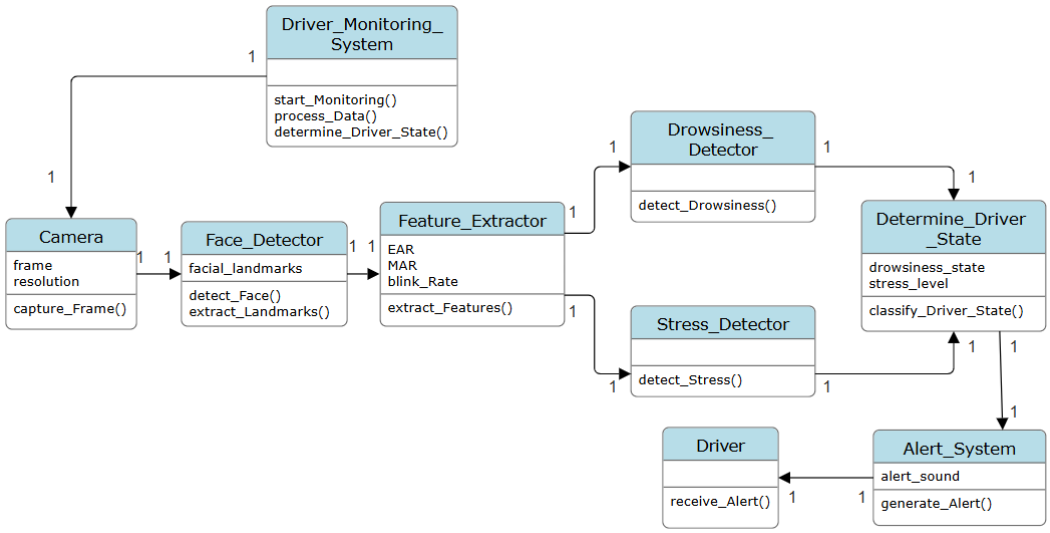


Fig 6.2.2: Class diagram

6.2.3 Sequence Diagram

The sequence diagram illustrates the step-by-step interaction between different components of the Driver Monitoring System. Initially, the driver starts the monitoring process, and the system captures video frames using the camera. These frames are passed to the face detection module, which identifies the face and extracts facial landmarks.

The extracted landmarks are then sent to the feature extraction module, where important features such as eye-related parameters (EAR) and facial muscle-based features are computed. These features are further processed by GNN-based models for drowsiness and stress detection. The models analyze the features and predict the respective levels.

The results from both detectors are returned to the Driver Monitoring System, which determines the overall driver state. Based on this state, the system generates an alert and displays a warning to the driver in real time, ensuring safety during driving.

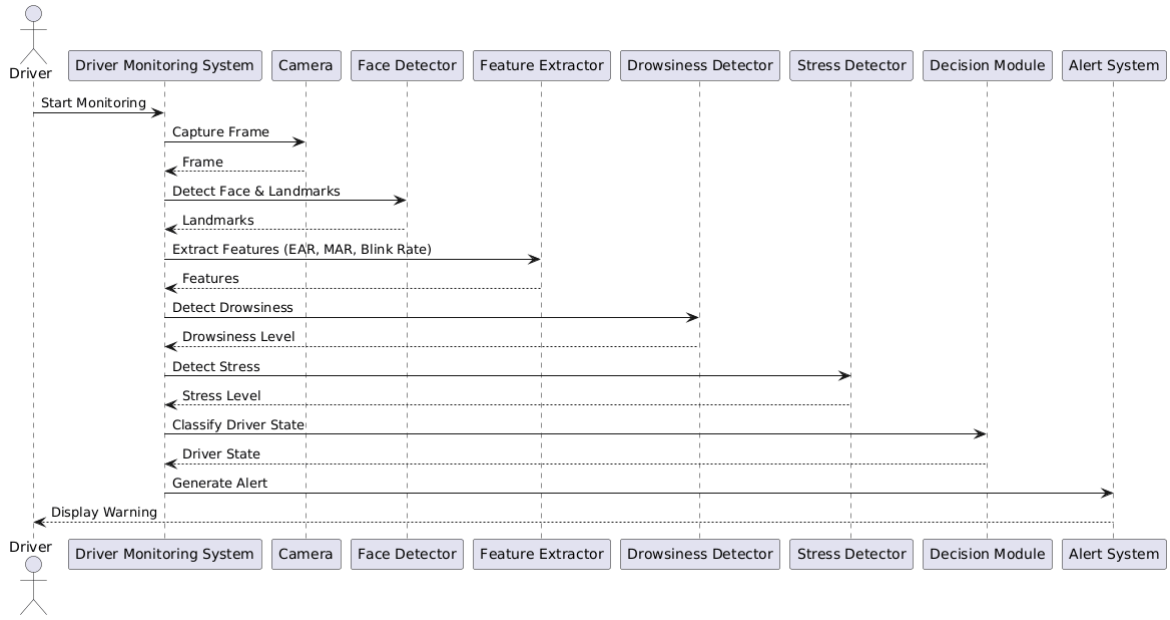


Fig 6.2.3: Sequence diagram

6.2.4 Activity Diagram

The activity diagram represents the workflow of the Driver Monitoring System from input to output. The process begins with starting the monitoring system, followed by capturing video frames using the camera. The system then detects the driver's face and extracts facial landmarks.

Based on these landmarks, important features such as eye aspect ratio (EAR), mouth aspect ratio (MAR) and blink rate are extracted. These features are processed using GNN-based models to detect drowsiness and stress. The system then determines the overall driver state.

If the driver is found to be drowsy, a drowsiness alert is generated. If the stress level is high, a stress alert is triggered. Otherwise, the driver is considered safe. Finally, the system displays a warning or alert message to ensure driver safety.

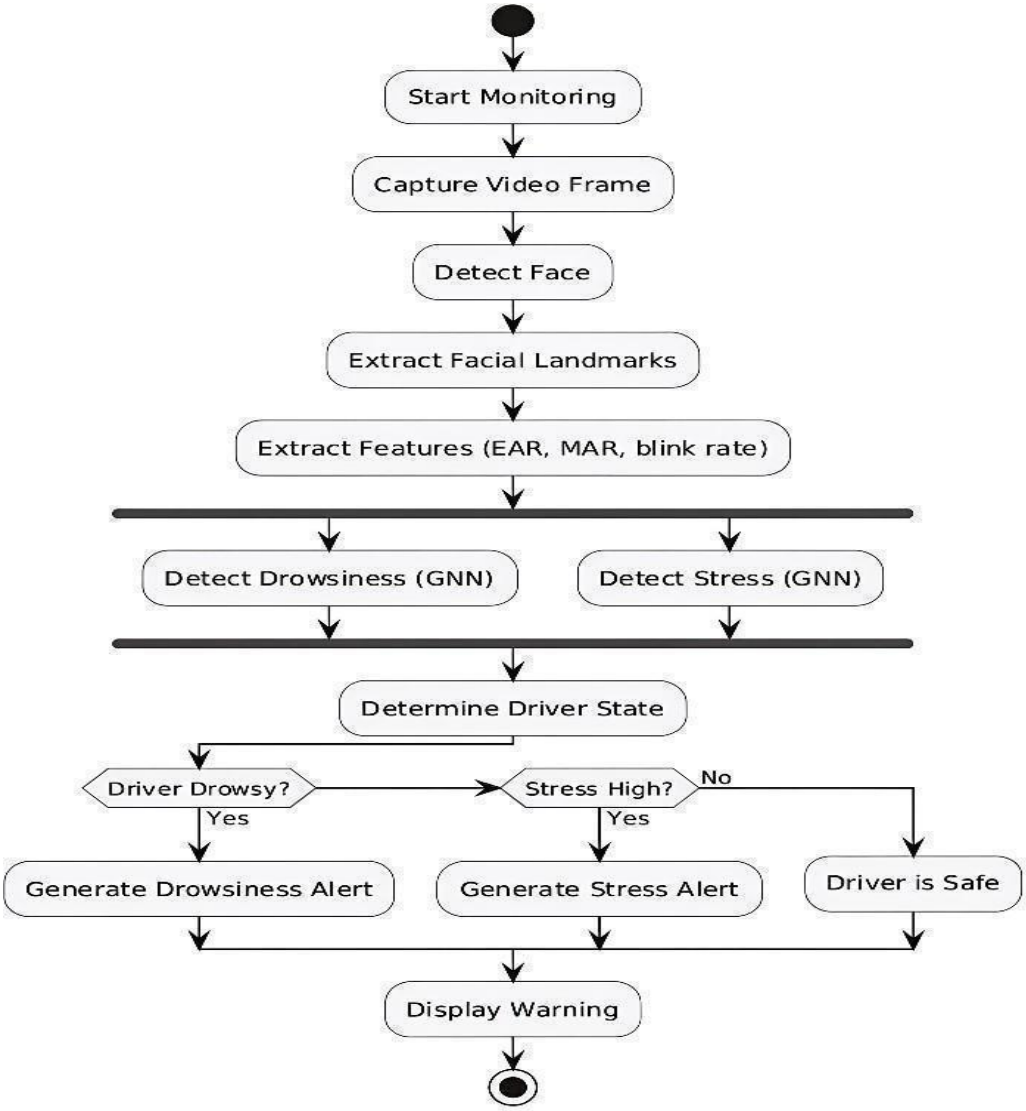


Fig 6.2.4: Activity Diagram

Chapter 7

SYSTEM IMPLEMENTATION

7.1 IMPLEMENTATION PROCESS

The implementation of the proposed driver monitoring system is carried out through a sequence of steps, starting from real-time video acquisition to intelligent decision-making using Graph Neural Networks (GNN). The system captures video through a webcam and processes each frame to analyze driver behavior.

- **Data Acquisition**

Video input is captured in real time using a webcam. Each frame is processed continuously to monitor the driver's facial features.

- **Face Detection and Landmark Extraction**

The system uses MediaPipe Face Mesh to detect the face and extract 468 facial landmarks. These landmarks represent key regions like eyes, mouth, and head position.

- **Feature Extraction**

From the detected landmarks, important features such as Eye Aspect Ratio (EAR), Mouth Aspect Ratio (MAR), and blink rate are computed to analyze driver behavior.

- **Drowsiness Detection**

The extracted facial landmark features are converted into a graph structure, where nodes represent landmarks and edges represents relationships. A Graph Neural Network (GNN) analyzes these relationships to detect whether the driver is ALERT or DROWSY.

- **Stress Detection**

Similarly, facial features are represented as a graph and passed through a GNN model trained to classify stress levels as LOW, MEDIUM, or HIGH.

- **Decision and Alert System**

The outputs from both GNN models are combined to determine the driver state (SAFE, WARNING, or CRITICAL), and alerts are generated in real time if necessary.

7.2 MODULE IMPLEMENTATION

The proposed driver monitoring system is implemented using multiple interconnected modules that work together to detect driver drowsiness and stress in real time. Each module performs a specific task to ensure accurate and efficient monitoring.

LIST OF MODULES:

1. Face & Landmark Detection Module
2. Classical Feature Extraction Module
3. Drowsiness Detection Module
4. Stress Detection Module
5. Feature Fusion & Decision Module

• **Face & Landmark Detection Module:**

This module captures real-time using webcam and uses MediaPipe Face Mesh to detect the driver's face and extract 468 facial landmarks. These landmarks represent important facial regions such as eyes, mouth, and head orientation.

• **Classical Feature Extraction Module:**

In this module, key features such as Eye Aspect Ratio (EAR), Mouth Aspect Ratio (MAR), and blink rate are calculated from facial landmarks. These features help in analyzing driver behavior.

• **Drowsiness Detection Module (GNN-Based):**

The extracted facial landmarks are converted into a graph structure, where nodes represent facial points and edges represent their relationships. A Graph Neural Network (GNN) processes this graph to classify the driver as ALERT or DROWSY.

Dataset Used: NTHU Driver Drowsiness (NTHU-DDD) Dataset

This dataset is used for training the drowsiness detection module. It contains images of drivers under conditions such as yawning, eye closure, head movement, and different lighting environments.

- **Stress Detection Module (GNN-Based):**

Similarly, facial features are modeled as a graph and passed into a GNN trained on stress related data. The model predicts the driver's stress level as LOW, MEDIUM, or HIGH.

Dataset Used: Cohn-Kanade (CK+) Dataset

The CK+ dataset is used for training the stress detection module. It includes facial expression sequences representing different emotional states, which are useful for identifying stress-related patterns.

- **Feature Fusion & Decision Module:**

This module combines the outputs from both drowsiness and stress detection. Based on the results, it determines the driver's state (SAFE, WARNING, or CRITICAL) and generates alerts when necessary.

Chapter 8

SYSTEM STUDY AND TESTING

8.1 FEASIBILITY STUDY

The feasibility of the proposed driver monitoring system is analyzed to ensure that the system is practical, efficient, and suitable for real-time implementation. The study evaluates whether the system can be developed using available resources and technologies.

Three key considerations involved in the feasibility analysis are:

1. Technical feasibility
2. Social feasibility
3. Economic feasibility

- **Technical Feasibility**

The proposed system is technically feasible as it uses standard technologies such as Python, OpenCV, and MediaPipe for face detection and landmark extraction. The implementation of Graph Neural Networks (GNN) for analyzing facial relationships is supported by existing libraries. The system runs on a normal computer with a webcam and does not require high-end hardware, making it easy to implement and deploy.

- **Social Feasibility**

The system is socially acceptable as it is non-intrusive and does not require any physical sensors attached to the driver. It improves road safety by continuously monitoring driver behavior and providing alerts when unsafe conditions are detected. The system is user-friendly and can be easily adopted in real-world driving environments.

- **Economic Feasibility**

The proposed system is cost-effective as it uses a simple webcam and open-source software tools. There is no need for expensive hardware like EEG sensors. The development and maintenance costs are low, making it affordable for practical applications.

8.2 SOFTWARE TESTING

Software testing is the process of evaluating the system to identify errors and ensure that it functions correctly according to the specified requirements. It helps in verifying the performance, accuracy, and reliability of the driver monitoring system under different real- time conditions. The testing process ensures that the system produces correct outputs for drowsiness and stress detection and generates alerts without failure.

Types of Testing

The following types of testing are performed in the proposed system:

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

1. **Unit Testing**

Unit testing is performed to verify the correctness of individual components of the system such as face detection, landmark extraction, feature computation (EAR, MAR), and GNN-based prediction modules. Each module is tested independently to ensure it produces accurate outputs for given inputs. For example, the facial landmark detector is tested to confirm correct extraction of key points, and the GNN models are tested to validate proper classification of drowsiness and stress levels. This helps in identifying errors at an early stage and improves overall system reliability.

2. **Integration Testing**

Integration testing is carried out to ensure that different modules of the system work together correctly. In this project, modules such as webcam input, face detection, feature extraction, GNN-Based drowsiness detection, and stress detection are integrated and tested as a complete pipeline. The data flow between modules is verified to ensure smooth communication and correct processing.

2. System Testing

System testing is performed on the complete integrated system to evaluate its performance under real-time conditions. The system is tested using live webcam input to verify its ability to detect driver drowsiness and stress simultaneously. Various scenarios such as normal driving, drowsy conditions, and different stress levels are tested. The system is also evaluated under different lighting conditions and head movements to ensure robustness, accuracy, and real-time performance.

3. Acceptance Testing

Acceptance testing is conducted to ensure that the system meets user requirements and performs as expected in real-world scenarios. The final system is tested by users to verify whether it correctly detects drowsiness and stress and generates appropriate alerts. The system is considered acceptable if it is easy to use, provides accurate results, and operates efficiently without requiring additional hardware. This testing confirms the practical usability and effectiveness of the proposed driver monitoring system.

Table 8.2.1: Test Cases for Driver Monitoring System

Test Case ID	Description	Input	Expected Output	Status
TC-01	System Start	Start webcam	System starts real-time monitoring	Passed
TC-02	Face Detection	Live face input	Face detected successfully	Passed
TC-03	Landmark Extraction	Face visible	468 facial landmarks extracted	Passed

TC-04	Feature Extraction	Eye & Mouth regions	EAR, MAR, blink rate computed	Passed
TC-05	GNN Graph Creation	Facial landmarks	Landmarks converted into graph structure	Passed
TC-06	Drowsiness Detection	EAR + landmark graph	Drowsiness detected (ALERT/DROWSY)	Passed
TC-07	Stress Detection	Facial landmarks + expressions	Stress level detected (LOW/MEDIUM/HIGH)	Passed
TC-08	Driver State Decision	Drowsiness + Stress outputs	Driver state determined (SAFE/WARNING/CRITICAL)	Passed
TC-09	Alert Generation	CRITICAL state	Alarm/warning triggered	Passed

Chapter 9

SOURCE CODE

Face_landmark_extraction.py

```
import cv2
import mediapipe as mp


mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    static_image_mode=False,
    max_num_faces=1,
    refine_landmarks=True
)

def  extract_landmarks(frame):
    rgb  =  cv2.cvtColor(frame,  cv2.COLOR_BGR2RGB)
    result = face_mesh.process(rgb)


    landmarks = []

    if result.multi_face_landmarks:
        for face_landmarks in result.multi_face_landmarks:
            for lm in face_landmarks.landmark:
                x = int(lm.x * frame.shape[1])
                y = int(lm.y * frame.shape[0])
                landmarks.append((x, y))


    return landmarks
```

Feature_extraction.py

```
import cv2
import mediapipe as mp
import  numpy as np

# Initialize MediaPipe

mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    static_image_mode=False,
    max_num_faces=1,
    refine_landmarks=True
)

# Blink variables

blink_count = 0
eye_closed = False
EAR_THRESHOLD = 0.2

# Distance function

def distance(p1, p2):
    return np.linalg.norm(np.array(p1) - np.array(p2))

# EAR

def     compute_EAR(landmarks):
    p1 = landmarks[33]
    p2 = landmarks[160]
    p3 = landmarks[158]
    p4 = landmarks[133]
    p5 = landmarks[153]
    p6 = landmarks[144]

    A = distance(p2, p6)
    B = distance(p3, p5)
    C = distance(p1, p4)

    return (A + B)/ (2.0 * C + 1e-6)

# MAR
def   compute_MAR(landmarks):
```

```
    p1  =  landmarks[13]
    p2  =  landmarks[14]
    p3  =  landmarks[78]
    p4 = landmarks[308]

    A = distance(p1, p2)
    C = distance(p3, p4)

    return A /(C + 1e-6)
```

Graph_creation.py

```
import torch
from torch_geometric.data import Data

def create_graph(landmarks):

    # Convert to tensor (nodes)
    x = torch.tensor(landmarks, dtype=torch.float)

    # Create edges (simple chain)
    edges = []

    for i in range(len(landmarks) - 1):
        edges.append([i, i + 1])
        edges.append([i+ 1, i]) # bidirectional

    edge_index = torch.tensor(edges, dtype=torch.long).t().contiguous()

    # Create graph object
    graph = Data(x=x, edge_index=edge_index)
    return graph
```

Drowsy_gnn.py

```
# GNN MODEL

class DrowsyGNN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = GCNConv(2, 32)
        self.conv2 =GCNConv(32, 64)
        self.conv3 = GCNConv(64,64)
        self.fc = torch.nn.Linear(64, 1)
    def forward(self, data):
```

```
        x, edge_index = data.x, data.edge_index

        x = F.relu(self.conv1(x,  edge_index))
        x = F.relu(self.conv2(x, edge_index))
        x = F.relu(self.conv3(x,  edge_index))

        x = torch.mean(x, dim=0) # global pooling

        x = self.fc(x)


        return torch.sigmoid(x)
```

Stress_gnn.py

```
#GNN   MODEL
class StressGNN(torch.nn.Module):
    def _init_(self):
        super().__ init__()
        self.conv1 = GCNConv(8, 64)
        self.conv2 = GCNConv(64,128)

        self.fc = torch.nn.Linear(128, 1)
    def  forward(self, data):
        x, edge_index =data.x, data.edge_index

        x = F.relu(self.conv1(x,  edge_index))

        x=F.dropout(x,p=0.3,training=self.training)


        x = F.relu(self.conv2(x,  edge_index))

        x = F.dropout(x,p=0.3,training=self.training)

        x = F.relu(self.conv3(x,  edge_index))

        x = torch.mean(x, dim=0)
        x = self.fc(x)
        return x
```

Final_main_code.py

```
import cv2
import torch
import  numpy as np
import mediapipe as mp
import winsound
```

```
import pyttsx3
import threading
import time

from torch_geometric.data import Data
fromdrowsy_gnn_train import DrowsyGNN
from stress_gnn_train import StressGNN
from graph_creation import  create_graph

# Load  Models
drowsy_model = DrowsyGNN()
drowsy_model.load_state_dict(torch.load("gnn_model_drowsy_new.pth", map_location="cpu"))
drowsy_model.eval()

stress_model = StressGNN()
stress_model.load_state_dict(torch.load("gnn_model_stress_new.pth", map_location="cpu"))
stress_model.eval()


# Voice System
engine = pyttsx3.init()
voices = engine.getProperty('voices')
engine.setProperty('voice', voices[1].id)
engine.setProperty('rate',120)

speech_lock = threading.Lock()

def  speak(text):
    with speech_lock:
        engine.stop()
        engine.say(text)
        engine.runAndWait()

# Alarm
alarm_on = False
alarm_event = threading.Event()

def  alarm():
    while not alarm_event.is_set):
        winsound.Beep(2000,100)
        time.sleep(0.05)
```

```
# MediaPipe

mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    static_image_mode=False,
    max_num_faces=1,
    refine_landmarks=True
)


# Webcam
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Camera not working")
    exit()

print("Camera opened")


# MAIN  LOOP

while True:

    ret, frame = cap.read()
    if not ret:
        continue

    frame = cv2.flip(frame,1)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    results = face_mesh.process(rgb)

    if results.multi_face_landmarks:

        landmarks = results.multi_face_landmarks[0].landmark

        # Create Features for GNN

        features_2 = []
        features_8 = []
```

```
for lm in landmarks:
    x = lm.x  -0.5
    y = lm.y  - 0.5

    features_2.append([x, y])
    features_8.append([x, y, x*y, x**2, y**2, 0.0, abs(x), abs(y)])

graph_drowsy = create_graph(features_2)
graph_stress = create_graph(features_8)

 # GNN Predictions

with torch.no_grad():
    d_out = drowsy_model(graph_drowsy).item()
    s_out = stress_model(graph_stress).item()


# Convert to Labels
drowsiness= "DROWSY"if d_out > 0.5 else "ALERT"

if s_out < 0.4:
    stress= "LOW"
elif s_out < 0.7:
    stress= "MEDIUM"
else:
    stress= "HIGH"

 # Driver State

if drowsiness== "DROWSY":
    driver_state = "CRITICAL"
elif stress == "HIGH":
    driver_state = "WARNING"
else:
    driver_state = "SAFE"

# Alert System
if driver_state == "CRITICAL":
    if not alarm_on:
        alarm_on = True
```

```
            alarm_event.clear()

            threading.Thread(target=alarm, daemon=True).start()

            threading.Thread(
                        target=speak,
                args=("Driver is drowsy. Please wake up immediately.",),
                daemon=True
            ).start()

        elif driver_state == "WARNING":
            alarm_on = False

            threading.Thread(
                target=speak,
                args=("Driver stress level is high. Pleasestay calm.",),
                daemon=True
            ).start()

        else:
            alarm_on = False
            alarm_event.set()

        # Display
        cv2.putText(frame, f"Drowsiness: {drowsiness}",
                (20,40), cv2.FONT_HERSHEY_SIMPLEX,
                0.9, (0,255,0), 2)

        cv2.putText(frame, f"Stress: {stress}",
                (20,80), cv2.FONT_HERSHEY_SIMPLEX,
                0.9, (128,0,128), 2)

        cv2.putText(frame, f"Driver State: {driver_state}",
                (20,120), cv2.FONT_HERSHEY_SIMPLEX,
                0.9, (0,0,255), 2)

    cv2.imshow("Driver Monitoring System", frame)

    if cv2.waitKey(1) &0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

Chapter 10

SCREEN LAYOUTS

Fig 10.1: Facial Landmark Detection using MediaPipe

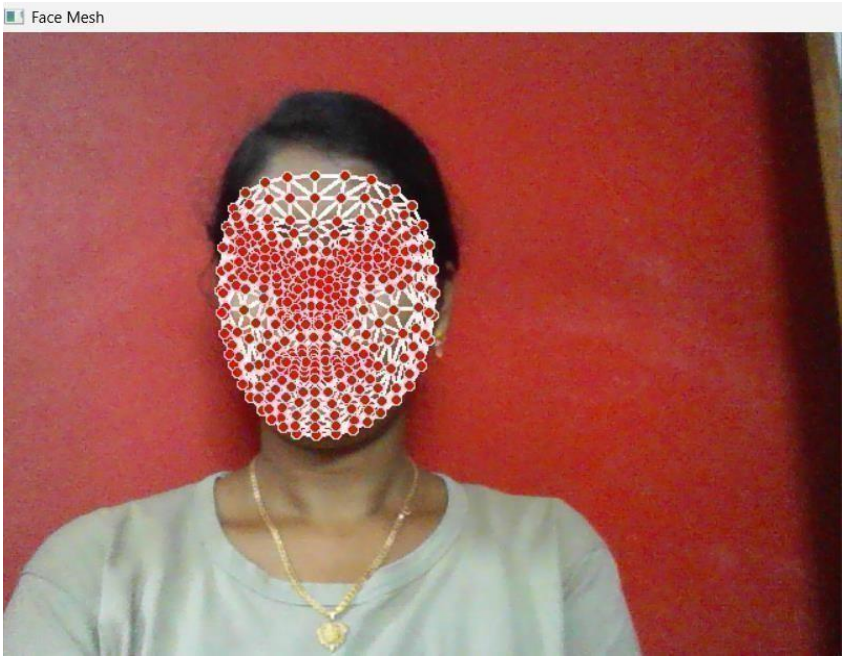


Fig 10.2: Feature Extraction

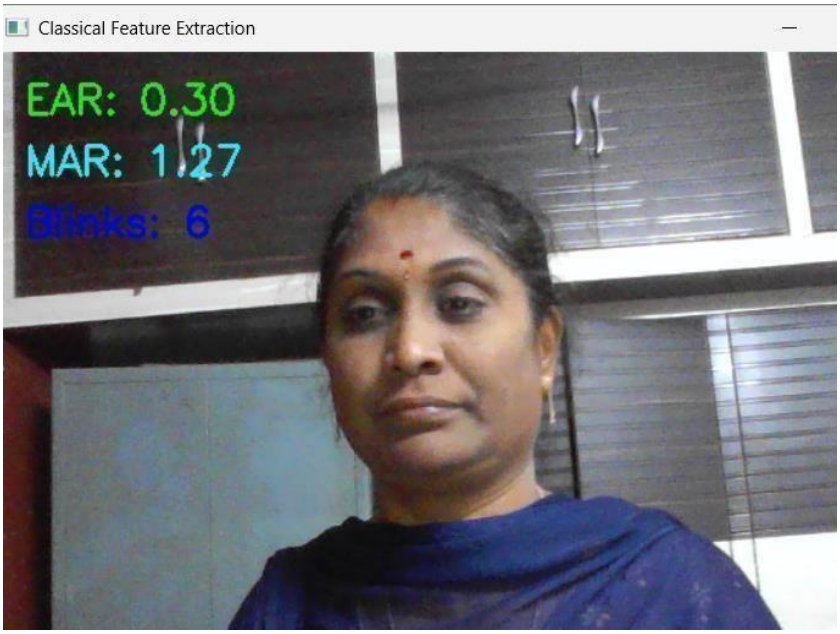


Fig 10.3.1: Drowsiness Detection (ALERT)

Fig 10.3.2: Drowsiness Detection (DOWSY)

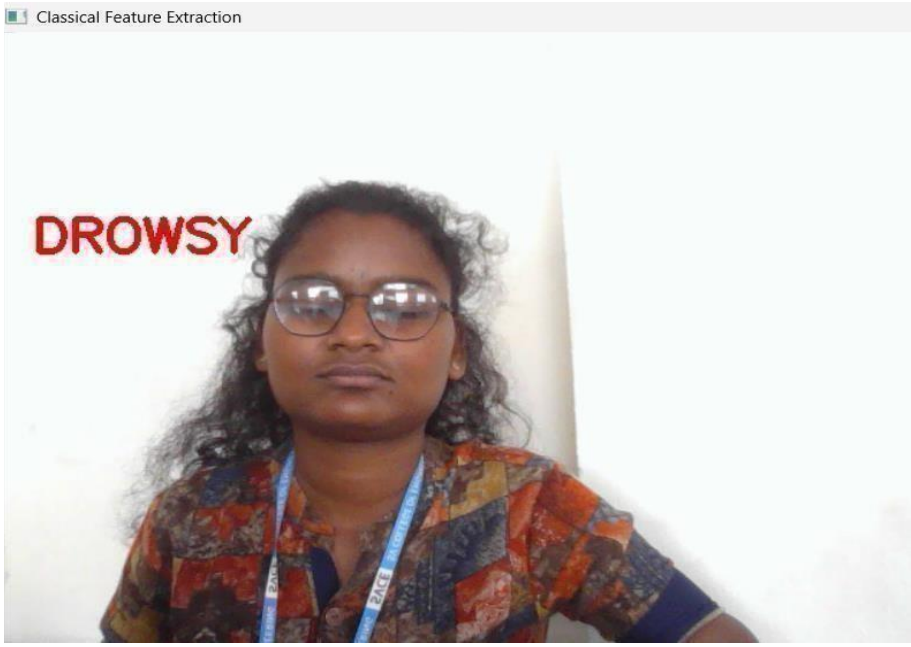


Fig 10.4.1: Stress Detection (LOW)

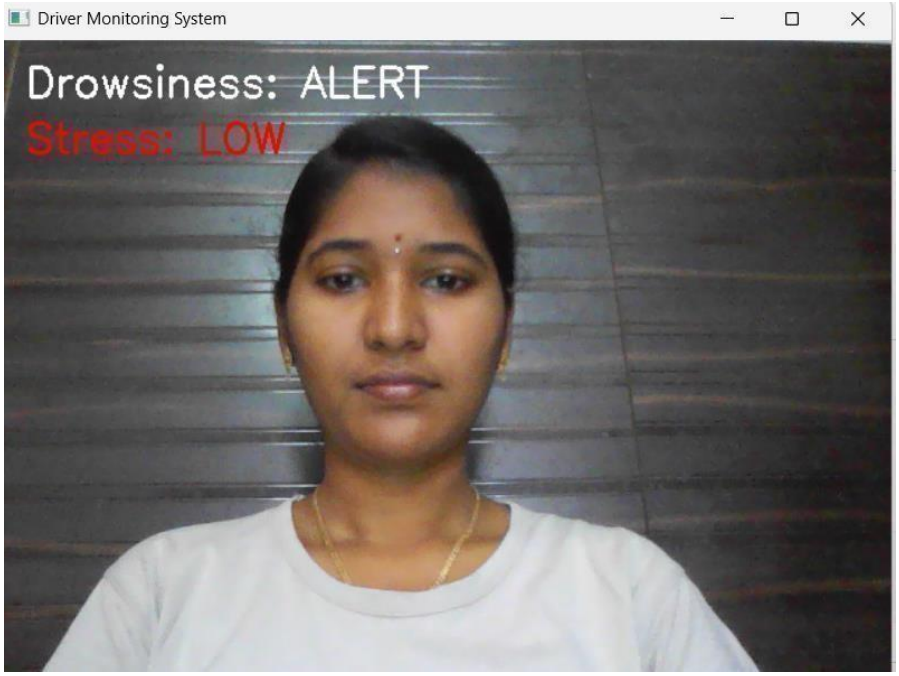


Fig 10.4.2: Stress Detection (MEDIUM)

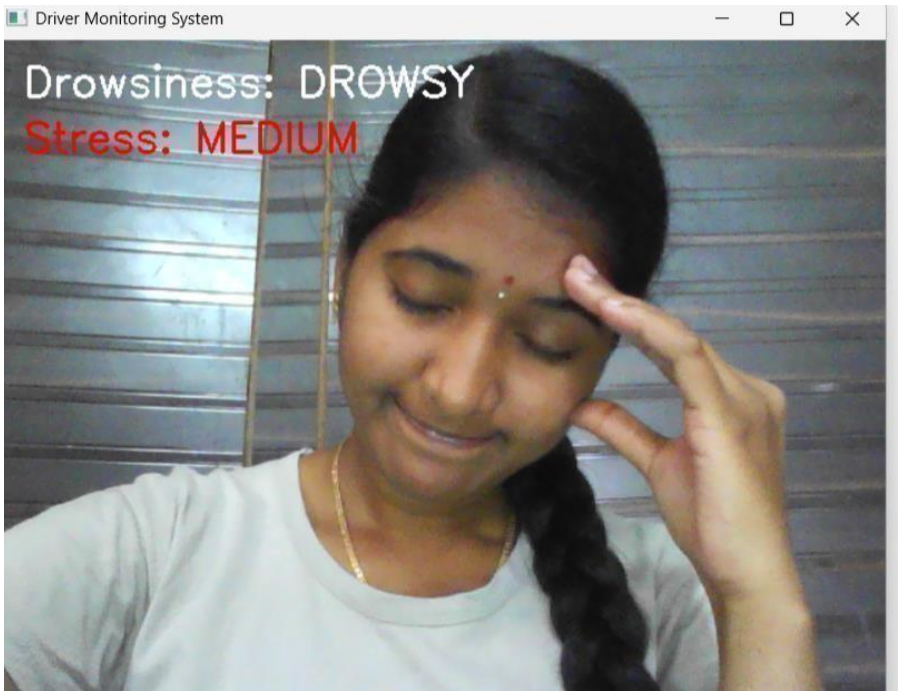


Fig 10.5.1: Final Driver State Classification-1

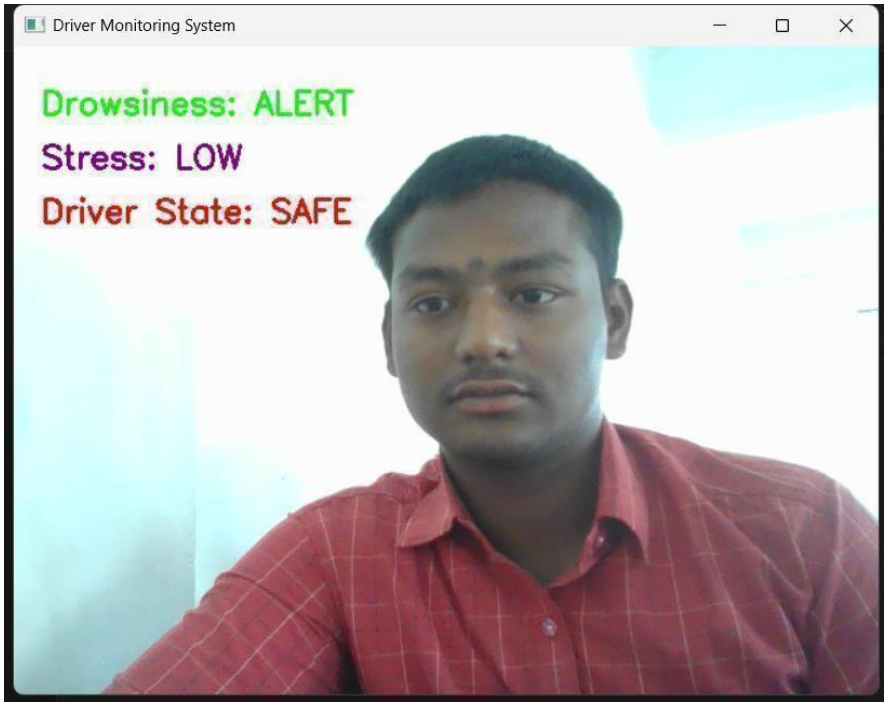
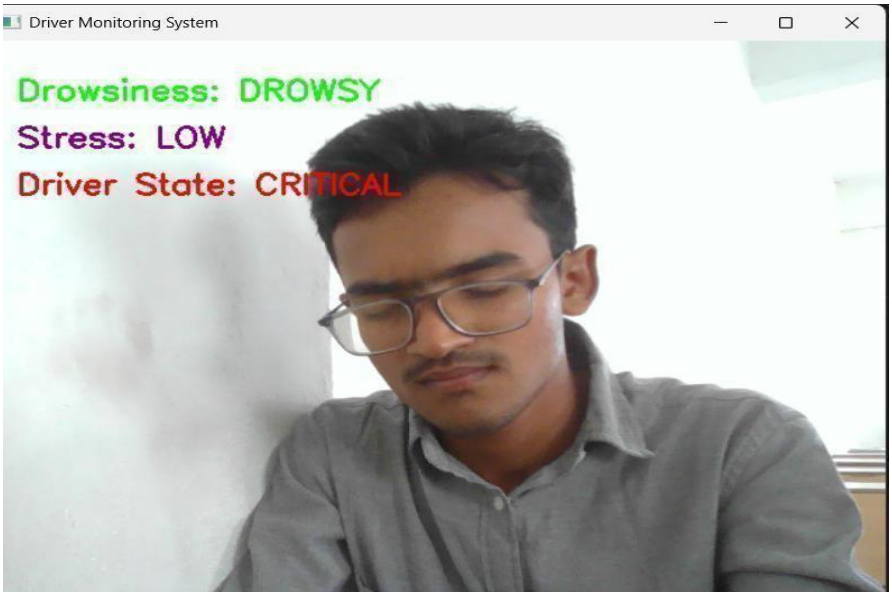


Fig 10.5.2: Final Driver State Classification-2



Chapter 11

FUTURE ENHANCEMENTS

The proposed driver monitoring system can be further enhanced by integrating it with vehicle control systems to enable automatic safety actions such as slowing down the vehicle or alerting nearby drivers in critical situations. The system accuracy can be improved by training the Graph Neural Network (GNN) model on larger and more diverse datasets that include variations in lighting conditions, facial expressions, and environmental factors.

In addition, the system can be extended to detect driver distractions such as mobile phone usage, head movement, and inattentive behavior, which are also significant causes of road accidents. For real-time deployment, the model can be implemented on embedded hardware platforms such as Raspberry Pi, making it suitable for in-vehicle applications.

Furthermore, mobile application integration or cloud -based monitoring can be incorporated to track driver status remotely and support fleet management systems. These enhancements will make the system more scalable, intelligent, and suitable for real-world applications.

Chapter 12

CONCLUSION

The proposed Hybrid Driver Monitoring System successfully detects both driver drowsiness and stress in real time using facial landmarks. The system utilizes Media Pipe Face Mesh to extract facial features such as EAR and MAR, while a Graph Neural Network (GNN) analyzes the relationships between these landmarks for improved behavioral understanding. By combining classical features with GNN-based analysis, the system provides accurate and reliable detection under real-world conditions. The model achieved a drowsiness detection accuracy of 94.41% and a stress detection accuracy of 95.11%, with an overall system accuracy of 94.60%. The final driver state is classified as SAFE, WARNING, or CRITICAL, and appropriate alerts are generated instantly. This approach enhances driving safety and helps in reducing accidents caused by driver fatigue and stress.

CHAPTER 13

REFERENCES

1 Varsha Agarwal, Rajneesh, "An Accurate Detection of Drowsiness Using a Graph-Based Neural Network," 2023 International Conference on Applied Intelligence and Sustainable Computing (ICAISC), 2023.

2 Neha Patil, Namrata Patole, Chetan Thorat, Abhishek Phapale, "Deep Learning Based Driver Yawning Detection," Journal of Emerging Technologies and Innovative Research (JETIR), 2023.

3 Quang Tran Ngoc, Seunghyun Lee, Byung Cheol Song, "Facial Landmark-Based Emotion Recognition via Directed Graph Neural Network."

4 Aryan Ritesh, Neel Jagatia, Pankaj Deshmukh, "Driver Drowsiness Detection Using Machine Learning," 2023 6th International Conference on Advances in Science and Technology (ICAST), 2023.

5 Md. Ebrahim Shaik, "A Systematic Review on Detection and Prediction of Driver Drowsiness," An article in Transportation Research Interdisciplinary Perspectives 21, 2023.

6 Dr. Harish S Gujjar, "A Deep Learning Approach to Automated Drowsiness Detection in Drivers Using CNN Models for Enhanced Road Safety," International Journal on Science and Technology (IJSAT), 2025.

7 Taejae Jeon, Hanbyeol Bae, Yongju Lee, Sungjun Jang, Sangyoun Lee, "Stress Recognition Using Face Images and Facial Landmarks," 2020 International Conference on Electronics, Information, and Communication (ICEIC), IEEE, 2020.

8 Mohammad Faizal, Nikhil, Prajwal, Aruna Rao B P, "Recognition of Stress Using Face Image and Facial Landmarks," International Advanced Research Journal in Science, Engineering and Technology, Vol. 8, Issue 5, May 2021.

9 Dr. R. M. S. Parvathi, M. Sanjay, P.S. Santhosh Kumar, P. Solai Murugan, "Stress Detection Using Machine Learning and Image Processing," International Journal of Creative Research Thoughts (IJCRT), 2025.

10 Vibin Varghese, Amritha Shenoy, Sreeram Ks, Remya K P, "EAR Based Driver Drowsiness Detection System," IOSR Journal of Engineering (IOSRJEN), 2018.

11 T. Soukupova and J. Cech, "Real-Time Eye Blink Detection Using Facial Landmarks," 21st Computer Vision Winter Workshop, 2016.

12 K. Dwivedi, K. Biswaranjan, and A. Sethi, "Drowsy Driver Detection Using Representation Learning," IEEE International Conference on Image Processing, 2014.